NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 7: Testing, Debugging, Exceptions, and Assertions

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 7 at a glance

Lecture 19	Lecture 20	Lecture 21
Testing and evidence Known answers, input classes, boundaries, black-box and glass-box testing, and regression.	**Systematic debugging** Error messages, tracebacks, controlled repairs, units, collections, and reproducible runs.	**Exceptions and assertions** Input contracts, expected failure, try/except, assertions, and an integrated report.

Specify → Test → Diagnose → Repair → Retest

Course specification: Session 7, Testing and Debugging; CLOs 1, 3, and 5.

NAZARBAYEV UNIVERSITY

LECTURE 19

Testing and Evidence of Correctness

Designing test cases that expose mistakes and support an engineering result

NAZARBAYEV UNIVERSITY

Lecture 19 learning goals

1. Specify inputs, units, outputs, and failure behavior.
2. Select known-answer, typical, boundary, and invalid cases.
3. Use black-box and glass-box approaches together.
4. Distinguish unit, integration, and regression testing.
5. Turn a prediction into a repeatable check.

Central question

What evidence would convince another engineer?

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Warm-up: two passing tests are not enough

```
# Deliberately faulty: the stated rule accepts
    exactly 2.0.
def classify_design(fos):
    if fos > 2.0:
        return "acceptable"
    return "not acceptable"

print(classify_design(2.5))
print(classify_design(1.5))
print(classify_design(2.0))
```

Stated rule

A design passes this *illustrative* check when $\mathrm{FoS} \geq 2.0$.

Predict first

Which call exposes a defect? What is the smallest repair?

Individual then pairs - 3 min

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Correct with respect to what?

Specification

- Input types and physical units.
- Valid ranges and assumptions.
- Returned result and accuracy.
- Expected response to invalid input.

Evidence

- A separately calculated answer.
- A boundary or failure case.
- A trace that explains behavior.
- Repeatable results from a clean run.

Two different questions

Does the code implement the specified model? Is that model appropriate for the real situation?

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Testing and debugging have different jobs

Testing

Run selected cases and compare actual behavior with a stated expectation. A failing test supplies a reproducible symptom.

Debugging

Investigate the symptom, identify its cause, repair the defect, and rerun tests.

A passing suite is evidence, not a universal proof

Untested inputs, incorrect expectations, and unsuitable model assumptions can still hide problems.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Write a test as an engineering experiment

Part	Question	Stress example
Input	What will be supplied?	$F = 10$ N; $A = 2$ m^2
Expected	What should happen?	$\sigma = 5$ Pa by hand calculation
Actual	What did the code return?	`calculate_stress(10, 2)`
Decision	Does it satisfy the criterion?	Compare values and units
Record	Can someone reproduce it?	Input, expectation, observation, outcome

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

Start with a hand-checkable case

```
# Inputs use N and m^2; the result is in Pa.
def calculate_stress(force_n, area_m2):
    return force_n / area_m2

expected_pa = 5.0  # Independent hand calculation: 10
      / 2.
actual_pa = calculate_stress(10.0, 2.0)
print("Expected:", expected_pa, "Pa")
print("Actual:", actual_pa, "Pa")
print("Pass:", actual_pa == expected_pa)
```

Why this case?

The expected answer is easy to obtain without running the function.

Scope

This first function assumes numeric inputs and a positive area. Explicit rejection is developed in Lecture 21.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Choose inputs for different reasons

Test class	Purpose	Example
Known-answer	Compare with an independent result	$10/2 = 5$ Pa
Typical	Exercise ordinary intended use	$F = 10\,000$ N; $A = 0.0005$ m^2
Boundary	Inspect limits and equality	FoS just below, at, above 2.0
Invalid input	Check documented rejection	Zero or negative area
Sanity check	Inspect scale, sign, and trend	Doubling area should halve stress

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Choose scope and repeatability deliberately

Test class	What is checked?	Example
Unit test	One function or small component	Area conversion by itself
Integration test	Components connected correctly	Conversion followed by stress calculation
Regression test	A previously repaired defect stays fixed	Keep the FoS $= 2.0$ test after repair

Complementary labels

One test can be a boundary test, a unit test, and a regression test at the same time.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Black-box testing: begin with the contract

Do not inspect the implementation yet

Construct cases from the promised behavior, valid domain, units, and boundary rules.

Example contract

Classify a finite temperature as too low, normal, or too high. Both endpoints of the supplied ordered interval are normal.

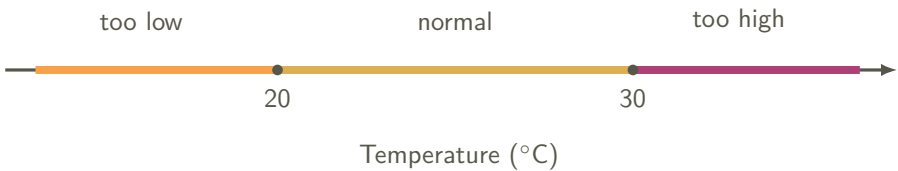
Your test-design question

Which values must be checked even before a function body exists?

Guttag, Section 6.1.1; Wang, Sections 8.2 and 8.7.

NAZARBAYEV UNIVERSITY

Partition the input space



Representative cases are not enough

Add the endpoints and values immediately on both sides of each endpoint.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

A boundary matrix makes the rule visible

Temperature (deg C)	Expected category	What it probes
19.9	too low	just below the lower limit
20.0	normal	inclusive lower limit
20.1	normal	just inside
29.9	normal	just inside
30.0	normal	inclusive upper limit
30.1	too high	just above the upper limit

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Glass-box testing: inspect the paths

```
# Contract: finite inputs; low_c <= high_c; limits
    are inclusive.
def classify_temperature(temp_c, low_c, high_c):
    if temp_c < low_c:
        return "too low"
    elif temp_c <= high_c:
        return "normal"
    return "too high"

print(classify_temperature(30.0, 20.0, 30.0))
```

Now inspect the body

Choose cases that enter every branch and exercise both outcomes of important conditions.

Coverage is not correctness

Executing every branch does not prove that each comparison or returned result is right.

Guttag, Section 6.1.2; original adaptation of the Week 4 classifier.

Combine contract tests with path tests

Black-box view

Does the result match the written promise? Test categories and boundaries from the specification.

Glass-box view

Which branches, loop exits, and recursive returns have actually been exercised?

Neither view replaces the other

A missing requirement may be invisible in the code. A rarely executed path may be missed by ordinary examples.

Guttag, Sections 6.1.1–6.1.2.

NAZARBAYEV UNIVERSITY

Test repetition: zero, one, and several passes

Structure	Minimum useful cases	Additional question
`for` loop	empty list; one item; several items	Does the final item contribute?
`while` loop	false initially; one update; several updates	Test every stopping route
Recursion	base case; one recursive step; deeper case	Does every call approach a base case?

Guttag, Section 6.1.2; Week 6 recursion test cases.

NAZARBAYEV UNIVERSITY

Activity: design tests that reveal the defect

```
# Deliberately faulty: the last measurement is
    omitted.
def total_load(loads_n):
    total_n = 0.0
    for index in range(len(loads_n) - 1):
        total_n += loads_n[index]
    return total_n

print(total_load([]))
print(total_load([5.0]))
print(total_load([2.0, 3.0, 4.0]))
```

Expected contract

Return the sum of all loads, in N. The empty list has sum 0.

Record three tests

Predict the outputs. Which tests fail? Repair the loop without changing the contract.

Pairs - 4 min

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Reuse Week 6: test the return path

```
# Valid input: a nonnegative integer; use small n
    here.
def factorial_recursive(n):
    if n == 0:
        return 1
    return n * factorial_recursive(n - 1)

# Base, near-base, and deeper known-answer tests.
print(factorial_recursive(0))
print(factorial_recursive(1))
print(factorial_recursive(4))
```

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

Three different paths

Zero recursive calls, one recursive call, and several recursive calls.

Agreement is supporting evidence

Two implementations can share the same misconception. Keep at least one independent expected answer.

NAZARBAYEV UNIVERSITY

Test floating-point results with a criterion

```
# Compare the computed result with an
    independent target.
actual = 0.1 + 0.2
expected = 0.3
absolute_tolerance = 1e-12

print(actual)
print(actual == expected)
print(abs(actual - expected) <=
    absolute_tolerance)
```

Absolute tolerance

$|x - x_{\text{expected}}| \leq \varepsilon$. The tolerance has the same unit as the compared quantity.

Do not hide model errors

This tiny tolerance checks arithmetic. It is not a statement about measurement accuracy.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Engineering trends are useful additional tests

For uniform axial stress, $\sigma(F, A) = F/A$ with $A > 0$:

$$\sigma(2F, A) = 2\sigma(F, A), \qquad \sigma(F, 2A) = \tfrac{1}{2}\sigma(F, A).$$

Also check

- Zero force gives zero stress.
- Signed force preserves the chosen sign convention.

But remember

A wrong constant unit multiplier can pass every scaling test. Check one absolute known answer too.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Test the data, not only the final scalar

Collections

- Empty, singleton, and multiple readings.
- First and last index behavior.
- Raw data unchanged when promised.

Dictionaries

- Existing and missing keys.
- Consistent units and field names.
- Missing information is not zero.

A useful assertion to plan

After a non-mutating calibration function, the original list must still equal its saved copy.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Integration catches mismatched interfaces



End-to-end known case

$10\,000$ N and 500 mm^2 must produce 20 MPa. Test the chain as well as each individual function.

Typical interface bug

Passing an area in mm^2 into a function that expects m^2 does not necessarily cause a Python error.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

A small repeatable test driver

```
# Use the classifier already defined in the previous
    example.
cases = [(19.9, "too low"), (20.0, "normal"),
         (30.0, "normal"), (30.1, "too high")]
passed = 0
for temp_c, expected in cases:
    actual = classify_temperature(temp_c, 20.0,
     30.0)
    ok = actual == expected
    print(temp_c, expected, actual, ok)
    if ok:
        passed += 1
print(f"Passed {passed}/{len(cases)} selected tests"
    )
```

Test driver

Supplies test cases, invokes the function, compares results, and reports evidence.

Readable report

Print the input, expectation, actual result, and pass status. Lecture 21 turns checks into assertions.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Activity: propose a five-case test plan

Contract

`mean_reading(values)` returns the mean of a nonempty list of finite readings. It does not modify its input. An empty list must be rejected.

Choose cases

Include a known answer, a singleton, equal values, mixed signs, and an empty input.

For every case

Write the expected result or rejection *before* executing the code. Include one non-mutation check.

Pairs - 4 min

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

Regression: keep the test after the repair

1. Capture the smallest input that exposed the defect.
2. State the expected result independently.
3. Repair one cause and rerun that test.
4. Rerun the existing suite for unintended changes.
5. Retain the new case for every later version.

Engineering record

A repair without a retained test can become the same investigation next week.

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

Lecture 19 exit ticket

New contract

`classify_pressure(p)` returns low for $p < 100$ kPa, normal for $100 \leq p \leq 200$ kPa, and high above 200 kPa.

1. Give one typical and two exact-boundary tests.
2. How would a glass-box test complement them?
3. Why does passing these tests not establish real-system safety?

Individual - 3 min

Wang, Ch. 8, Sections 8.2 and 8.7–8.10; Guttag, Section 6.1.

NAZARBAYEV UNIVERSITY

LECTURE 20

Systematic Debugging and Error Messages

Using tracebacks, values, units, and controlled experiments to locate defects

NAZARBAYEV UNIVERSITY

Lecture 20 learning goals

1. Distinguish syntax, runtime, logic, unit, data, and numerical errors.
2. Read a traceback from symptom back to source.
3. Inspect values, types, units, and control flow.
4. Reduce a failing example and change one cause at a time.
5. Turn a repair into documented regression evidence.

Central question

Where does the program first disagree with the expected state?

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Warm-up: a short script can hide several defects

```
# Deliberate debugging exercise: name and unit
    defects.
Force = 10000.0              # N
area_mm2 = 500.0             # mm^2
yield_strength_mpa = 250.0   # MPa
area_m2 = area_mm2 / 1000    # Inspect this conversion
    .
stress_pa = force / area_m2
fos = yield_strength_mpa / stress_pa
print(f"Stress = {stress_pa:.2f} Pa")
print(f"FoS = {fos:.5f}")
```

Before running

Find one name problem and two unit problems.

Predict the intended result

$F = 10\,000$ N; $A = 500$ mm^2; strength $= 250$ MPa. Estimate stress and FoS.

Pairs - 3 min

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Classify the symptom before repairing it

Category	What happened?	First useful action
Syntax	Python cannot parse this code	Check punctuation and indentation
Runtime	A statement raises an exception	Read the traceback and inspect inputs
Logic	Code runs but violates its specification	Trace the first wrong state
Unit / data	Quantity or record means something else	Check conversions, schema, provenance
Numerical	Approximation misses the requirement	Check tolerance and stopping evidence

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Syntax errors: inspect the structure

```
# Deliberately invalid Python: a colon is
    missing.
stress_mpa = 120.0
if stress_mpa <= 125.0
    print("Within the stated limit")
```

Typical checks

- Header ends in a colon.
- Parentheses and quotes match.
- Indentation forms valid blocks.

Detection is a clue

The marked location is where parsing failed; the cause may be just before it.

Wang, Section 8.3.1; Python Tutorial, Section 8.1.

NAZARBAYEV UNIVERSITY

A runtime failure preserves a call trail

```
# Deliberate failure used to inspect the call
    path.
def calculate_stress(force_n, area_m2):
    return force_n / area_m2

def report():
    print(calculate_stress(10000.0, 0.0))

report()
```

Predict the failure

Which operation fails? Which function called it? Which argument caused the invalid operation?

Useful context

The arithmetic line reports the symptom. The caller supplies the zero denominator.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Read a traceback from the bottom upward

```
Traceback (most recent call last):
  File "traceback_demo.py", line 8, in <module>
    report()
  File "traceback_demo.py", line 6, in report
    print(calculate_stress(10000.0, 0.0))
  File "traceback_demo.py", line 3, in calculate_stress
    return force_n / area_m2
ZeroDivisionError: float division by zero
```

Traceback excerpt: path prefixes and version-dependent caret marks omitted.

Wang, Section 8.4; Python Tutorial, Section 8.2.

Reading order

1. Exception type and message.
2. Failing operation and line.
3. Caller and supplied values.

NAZARBAYEV UNIVERSITY

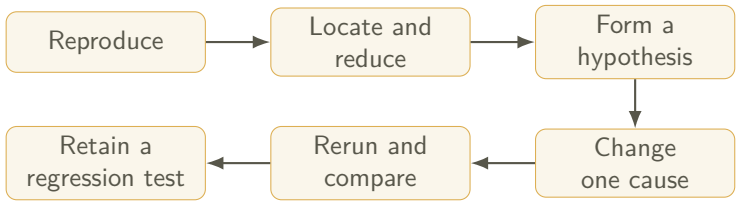
Common messages are diagnostic categories

Exception	Typical meaning here	Inspect
`NameError`	A name is unavailable	Spelling, case, scope, call order
`TypeError`	An operation received an unsuitable type	Value and type; missing return
`ValueError`	A value is unacceptable	Conversion format or valid range
`IndexError`	A sequence position does not exist	Index and collection length
`KeyError`	A dictionary key is absent	Keys, spelling, nesting path

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Debugging is a controlled investigation



Do not edit at random

A change is useful when a hypothesis predicts how the observation should change.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Reproduce, then make the example smaller

Keep the evidence

- Exact input and failing output.
- The code version and run order.
- Expected result and its basis.

Remove unrelated work

- Use one record, not the whole dataset.
- Separate calculation from printing.
- Temporarily remove plotting or saving.

A useful minimal example

Small enough to inspect completely, but still reproduces the same failure.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Inspect values, types, and invisible characters

```
# Inspect the raw value before choosing a
    conversion.
area_text = " 500 "
print("Raw:", repr(area_text))
print("Type:", type(area_text))

# Convert only after confirming the field means
     area in mm^2.
area_mm2 = float(area_text)
print("Converted:", area_mm2, "mm^2")
```

Three tools

`print(value)` shows a value.
`type(value)` identifies its type.
`repr(text)` exposes surrounding whitespace and escapes.

Meaning still matters

A successful conversion does not prove that the field has the intended unit.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Repair 1: the script runs, but is it right?

```
# Name defect repaired; both unit defects remain
    deliberately.
force_n = 10000.0
area_mm2 = 500.0
yield_strength_mpa = 250.0
area_m2 = area_mm2 / 1000
stress_pa = force_n / area_m2
fos = yield_strength_mpa / stress_pa

# Inspect intermediate values before trusting the
    final label.
print("Area:", area_m2, "m^2")
print("Stress:", stress_pa, "Pa")
print("FoS:", fos)
```

Observed after the name fix

Area $= 0.5$ m^2; stress $= 20\,000$ Pa; FoS $= 0.0125$.

First wrong intermediate state

Does 500 mm^2 really equal 0.5 m^2?

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Repair 2: derive the area conversion

$$1\ \text{mm} = 10^{-3}\ \text{m} \quad \Longrightarrow \quad 1\ \text{mm}^2 = 10^{-6}\ \text{m}^2.$$

$$500\ \text{mm}^2 = 0.0005\ \text{m}^2, \qquad \sigma = \frac{10\,000}{0.0005} = 20 \times 10^6\ \text{Pa}.$$

Correct expression

```
area_m2 = area_mm2 / 1_000_000
```

One defect still remains

Material strength in MPa cannot be divided directly by stress in Pa to obtain a dimensionless FoS.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Repair 3: align units before taking a ratio

```python
# A complete repair uses consistent SI units
    internally.
force_n = 10000.0
area_mm2 = 500.0
yield_strength_mpa = 250.0
area_m2 = area_mm2 / 1_000_000
strength_pa = yield_strength_mpa * 1_000_000
stress_pa = force_n / area_m2
fos = strength_pa / stress_pa

print(f"Stress = {stress_pa / 1e6:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
```

Independent checks

$10\,000/500 = 20$ N/mm^2 $= 20$ MPa.
$250/20 = 12.5$.

Scope of the result

Uniform axial stress and the supplied strength only; not a complete component-safety assessment.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

A debugging log connects changes to evidence

Observed symptom	Hypothesis / repair	New evidence
NameError	Use one consistent force name	Execution reaches calculation
Area 0.5 m^2	Square the length-unit conversion	Area 0.0005 m^2
Implausible FoS	Align strength and stress units	Dimensionless FoS $=$ 12.5
Future risk	Retain known-answer regression	20 MPa and 12.5 checked again

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Trace a state variable, not just the output

```
# Deliberate logic bug: the total is reset on
    every pass.
powers_kw = [1.0, 2.0, 3.0]
interval_h = 0.5
for power_kw in powers_kw:
    total_kwh = 0.0
    total_kwh += power_kw * interval_h
    print("Intermediate:", total_kwh)
print("Final:", total_kwh, "kWh")
```

Expected result

$(1 + 2 + 3) \times 0.5 = 3$ kWh.

State question

Where should the accumulator be initialized? What happens with an empty input list?

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Trace a None back to its source

```
# Deliberate defect: a printed value is not
    returned.
def calculate_current(voltage_v, resistance_ohm
    ):
    current_a = voltage_v / resistance_ohm
    print(current_a)

current_a = calculate_current(12.0, 4.0)
print("Stored:", current_a)
power_w = 12.0 * current_a
```

Predict the sequence

A number is printed. What is stored by the caller? Which later expression fails?

Minimal repair

Return the current from the calculation function; print it in the reporting layer.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Debugging collections: watch for side effects

```
# Deliberate side-effect bug: assignment does not
    copy a list.
def corrected_readings(raw_c, offset_c):
    processed_c = raw_c
    for i in range(len(processed_c)):
        processed_c[i] -= offset_c
    return processed_c

raw_c = [22.0, 23.0]
result_c = corrected_readings(raw_c, 0.5)
print("Raw:", raw_c)
print("Processed:", result_c)
```

Required behavior

Return corrected temperatures without modifying the original readings.

Locate the cause

Which assignment creates an alias? What saved-state test would expose the violation?

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Debug dictionaries one lookup at a time

```
# Deliberate lookup mismatch in a controlled
    name field.
materials = {"steel": {"density_kg_m3":
    7850.0}}
requested = " Steel "
print("Requested:", repr(requested))
print("Known names:", list(materials.keys()))
record = materials[requested]
print(record["density_kg_m3"])
```

First lookup

Inspect the material name before accessing its property record.

Second lookup

Then check that the property key and expected unit exist.

Do not fabricate data

Replacing missing density with 0.0 can conceal the failure.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

When a loop does not finish

1. Identify the variable used by the stopping condition.
2. Trace its value across two or three passes.
3. Check that the update moves toward stopping.
4. Inspect tolerance, bounds, and work limits.
5. Stop a stuck run; reproduce with a small input.

No false success

A maximum-iteration limit prevents endless work; it does not turn the last iterate into a valid solution.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

A clean start reveals hidden state

Notebook risk

A value left by an earlier cell can make an incomplete notebook appear to work.

Before sharing

- Restart the kernel or interpreter.
- Run from the first cell or script line.
- Check that every input is defined.
- Reproduce outputs without manual repair.

Reproducibility test

A peer should not need your previous session to reproduce the result.

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Activity: diagnose an entire tiny program

Use the lab starter

Open `debug_lab_starter.py`. It has a returned-value defect, a unit-conversion defect, and an inclusive-boundary defect.

1. Run the starter and identify the first symptom.
2. Predict the effect of one repair before applying it.
3. Reach the hand benchmark: 125 MPa and FoS $= 2.0$.
4. Add one regression test for each repaired cause.

Pairs - 6 min

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

Ask for useful help, including from AI tools

Provide evidence

- A minimal reproducible example.
- Exact message or unexpected output.
- Expected behavior, input, and units.
- The hypothesis already tested.

Keep responsibility

- Request an explanation, not a blind rewrite.
- Review every changed line.
- Retest the original failure.
- Follow the permitted-assistance rules.

Protect information

Remove personal data, confidential records, and credentials before sharing an example.

Teaching extension: code review and responsible assistance; Week 6 accountability slide.

Lecture 20 exit ticket

1. What useful information is on the last traceback line?
2. Why can the cause lie earlier than the failing statement?
3. Explain one unit defect from the bar example.
4. What must be done after the repaired example passes?

One-sentence summary

Debugging replaces random edits with a reproducible explanation of cause and effect.

Individual - 3 min

Wang, Ch. 8, Sections 8.3–8.6 and 8.14–8.18; Guttag, Section 6.2.

NAZARBAYEV UNIVERSITY

LECTURE 21

Exceptions, Assertions, and Reliable Programs

Reporting invalid inputs honestly and making expectations executable

NAZARBAYEV UNIVERSITY

Lecture 21 learning goals

1. Raise an exception for a violated input contract.
2. Handle an expected failure with a narrow try/except block.
3. Trace the normal and exceptional paths.
4. Use assertions for tests and internal expectations.
5. Combine validation, computation, and evidence in a small program.

Central question

How should the program fail when it cannot produce a justified result?

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Warm-up: what should invalid area produce?

The request

An axial-stress function receives $F = 100$ N and $A = 0$ m^2.

Proposed responses

- Return 0 Pa.
- Return a huge arbitrary stress.
- Return None.
- Raise a clear error.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

Discuss

Which response best prevents an invalid input from being mistaken for an engineering result?

Pairs - 3 min

NAZARBAYEV UNIVERSITY

An exception signals an abnormal path

Normal path

Receive valid inputs, calculate the requested quantity, and return its value.

Exceptional path

Stop the current operation and transfer control to a matching handler, or report an unhandled exception.

The goal is not to suppress every error

Report or recover from expected failures. Unexpected failures should remain visible.

Wang, Section 8.12; Python Tutorial, Sections 8.2–8.3.

NAZARBAYEV UNIVERSITY

Raise an error before invalid arithmetic

```
# Core contract: finite numeric inputs; signed force
    is allowed.
def calculate_stress(force_n, area_m2):
    """Return uniform axial stress in Pa from N and
    m^2."""
    if area_m2 <= 0:
        raise ValueError("Area must be positive, in
    m^2.")
    return force_n / area_m2

print(calculate_stress(10.0, 2.0))
```

Read the contract

Valid area is strictly positive. The failure is now a deliberate ValueError, not an accidental division by zero.

Stated scope

Finite numeric inputs are assumed here; the integrated example also checks finiteness.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Choose an exception that explains the problem

Situation	Useful exception	Example
Wrong kind of value	`TypeError`	Arithmetic receives a list instead of a number
Unacceptable value	`ValueError`	Area is zero or negative
Missing required field	`KeyError`	Required property absent
Failed internal/test claim	`AssertionError`	Actual result violates a test

A useful message

Name the quantity, state the requirement, and include the expected unit.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Handle an expected conversion failure

```
# Simulate a user-entered field without an
    interactive prompt.
text = "not a number"
try:
    force_n = float(text)
    print("Converted force:", force_n)
except ValueError:
    print("Force must be a numerical value in N
    .")
print("Input check finished")
```

try

Execute the protected operation.

except ValueError

Handle this particular failure. Statements after the failing conversion inside try are skipped.

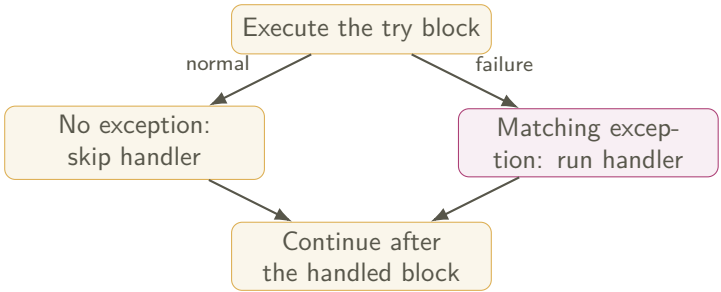
Afterward

Continue after the handled block.

Wang, Section 8.12.2; Python Tutorial, Section 8.3.

NAZARBAYEV UNIVERSITY

Trace the two control-flow routes



Without a matching handler, the exception propagates outward instead.

Python Tutorial, Section 8.3; original control-flow diagram.

NAZARBAYEV UNIVERSITY

Activity: predict the printed letters

```
# Predict the route before changing the input
    text.
text = "bad"
try:
    print("A")
    value = float(text)
    print("B")
except ValueError:
    print("C")
print("D")
```

Two runs

Predict the output for text = "bad" and for text = "12.5".

Explain

Which line is skipped in each run? Where does execution continue?

Individual - 3 min

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Different failures can need different responses

```
# One block can distinguish several expected failures
    .
text = "0"
try:
    resistance_ohm = float(text)
    current_a = 12.0 / resistance_ohm
except ValueError:
    print("Resistance must be numerical.")
except ZeroDivisionError:
    print("Resistance cannot be zero.")
else:
    print(f"Current = {current_a:.2f} A")
```

Python Tutorial, Section 8.3; teaching example using the voltage-resistance relationship.

Try three fields

"0": division failure.
"bad": conversion failure.
"4": normal result.

Scope

This small trace example is not a complete positive-resistance validator.

Use else for the successful route

```python
# Keep the protected operation narrow.
text = "12.5"
try:
    force_n = float(text)
except ValueError:
    print("Force must be numerical.")
else:
    # This runs only after the conversion
    succeeds.
    print(f"Accepted field: {force_n:.1f} N")
```

Python Tutorial, Section 8.3.

Why separate it?

The successful follow-up is outside the protected conversion block.

Important

An exception raised inside this else block is not caught by the preceding except clause.

A caller can handle a function error

```
# Use calculate_stress from the validated function
    example.
try:
    stress_pa = calculate_stress(100.0, 0.0)
except ValueError as error:
    # Preserve the function's explanation for the
    user.
    print(f"Cannot calculate stress: {error}")
else:
    print(f"Stress = {stress_pa:.2f} Pa")
```

Function responsibility

Reject an input that violates the calculation contract.

Caller responsibility

Choose whether to explain, retry, or stop. Do not print a result when no valid result exists.

Wang, Sections 8.12.1–8.12.2; Python Tutorial, Section 8.3.

NAZARBAYEV UNIVERSITY

Combine a retry loop with a clear exit

```
# Retry bad input; q exits deliberately.
while True:
    text = input("Area in m^2 (or q): ").strip()
    if text.lower() == "q":
        print("Cancelled")
        break
    try:
        area_m2 = float(text)
    except ValueError:
        print("Enter a number.")
        continue
    if area_m2 <= 0:
        print("Area must be positive.")
        continue
    print("Accepted:", area_m2, "m^2")
    break
```

Three paths

Cancel; reject and retry; accept and stop.

Preview limitation

The range check assumes a finite number. Additional numeric-data checks are shown later.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Do not convert failure into plausible data

```python
# Deliberately bad design: do not turn every
    failure into zero.
def hidden_failure(force_n, area_m2):
    try:
        return force_n / area_m2
    except Exception:
        return 0.0

print(hidden_failure(100.0, 0.0))
```

Why this is misleading

The caller receives 0.0 Pa even though no valid stress was calculated.

Better policy

Validate the domain; catch only failures you can explain or recover from. Let unexpected defects remain visible.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Assertions make expectations executable

```
# An assertion encodes an expected result
    during testing.
actual_pa = calculate_stress(10.0, 2.0)
expected_pa = 5.0
assert actual_pa == expected_pa, "Known stress
    case failed"
print("Known-answer check passed")
```

When the condition is true

Execution continues without an assertion message.

When the condition is false

Python raises AssertionError with the supplied explanation.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

A test function is still just a function

```
# Group repeatable checks in a small ordinary
    function.
def test_calculate_stress():
    assert calculate_stress(10.0, 2.0) == 5.0
    assert calculate_stress(0.0, 2.0) == 0.0
    assert calculate_stress(-10.0, 2.0) == -5.0
    # Separate unit-conversion benchmark for
    realistic inputs.
    actual_mpa = calculate_stress(10000.0, 0.0005) /
     1e6
    assert abs(actual_mpa - 20.0) <= 1e-9

test_calculate_stress()
print("Four selected stress checks passed")
```

Coverage of this small suite

Known answer; zero force; signed force; realistic units.

Still add

Invalid-area rejection and boundary cases in any later classification.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

Test that a function rejects invalid input

```
# A successful invalid-input test observes the
    specified error.
try:
    calculate_stress(10.0, 0.0)
except ValueError:
    print("Passed: zero area was rejected")
else:
    raise AssertionError("Expected ValueError
    for zero area")
```

Expected behavior

For zero area, rejection is the correct result of the test.

Unexpected outcomes

No exception: fail the test. A different exception: investigate instead of treating it as success.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

A tolerance is part of the test specification

```
# Test numerical agreement using an explicit
    tolerance.
actual_k = 25.0 + 273.15
expected_k = 298.15
absolute_tolerance_k = 1e-9
assert abs(actual_k - expected_k) <=
    absolute_tolerance_k

# The same idea handles a familiar floating-
    point example.
assert abs((0.1 + 0.2) - 0.3) <= 1e-12
print("Two tolerance checks passed")
```

State the quantity

Temperature difference in K; not an unexplained number of displayed decimal places.

Separate uncertainties

A numerical test tolerance is not automatically a sensor accuracy or model-error bound.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Validation is not an assertion

Purpose	Use	Why
External or caller input	`if` with `raise`	Required contract checks stay explicit
Known-answer test	`assert actual == expected`	Makes a development expectation executable
Internal invariant	`assert condition, "message"`	Exposes an unexpected internal state

Critical distinction

Python can omit assert statements in optimized execution (`python -O`). Never use them as the only protection for bad user input.

Wang, Section 8.11; Python Language Reference, Section 7.3.

NAZARBAYEV UNIVERSITY

An internal invariant checks our own logic

```
# Readings are already valid; check the internal
    accounting.
readings_c = [19.0, 22.0, 31.0]
normal_count = 0
warning_count = 0
for value_c in readings_c:
    if 20.0 <= value_c <= 30.0:
        normal_count += 1
    else:
        warning_count += 1
# Every reading must enter exactly one of these two
    groups.
assert normal_count + warning_count == len(readings_c)
print(normal_count, warning_count)
```

Teaching adaptation of Wang, Section 8.11: assertions as internal checks.

Invariant

Normal count + warning count = total readings.

Not input validation

This checks that our program accounted for every reading. It does not prove that the measurements are valid.

NAZARBAYEV UNIVERSITY

Activity: finish the mean-reading test suite

```
# Contract: a list of finite numerical readings, not
    modified.
def mean_reading(values):
    if len(values) == 0:
        raise ValueError("At least one reading is
    required.")
    return sum(values) / len(values)

readings = [2.0, 4.0, 6.0]
before = readings.copy()  # Preserve evidence for a
    side-effect test.
assert mean_reading(readings) == 4.0
assert readings == before
print("Mean and non-mutation checks passed")
```

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

Complete the plan

Add a singleton, equal values, mixed signs, and an expected ValueError for an empty list.

Check behavior

Also verify that the original list remains unchanged.

Pairs - 4 min

NAZARBAYEV UNIVERSITY

Conversion is not complete validation

```
# Some strings convert to floats without
    representing finite data.
import math
for text in ["12.5", "nan", "inf"]:
    value = float(text)
    print(text, math.isfinite(value))
```

New library tool

`math.isfinite(value)` is True for a finite number and False for NaN or infinity.

Engineering decision

The integrated stress report accepts finite inputs only. Check the domain after parsing.

Additional numeric-data safeguard; Python math reference: isfinite.

NAZARBAYEV UNIVERSITY

Integrated stress report: contract and evidence

Input contract

- Finite numerical force in N.
- Positive finite area in mm^2.
- Positive allowable stress in MPa.
- Signed stress; compare magnitude.

Output and failure

- Return stress and a stated-rule status.
- Reject invalid inputs explicitly.
- Do not print a PASS after failure.
- Keep input, calculation, and output separate.

Hand benchmark

$10\,000/80 = 125$ N/mm^2 $= 125$ MPa; equality passes a 125 MPa limit.

Integrated teaching example adapting Wang, Sections 8.12–8.14.

NAZARBAYEV UNIVERSITY

Integrated stress report: calculation and decision

```
    if area_mm2 <= 0 or allowable_mpa <= 0:
        raise ValueError("Area and allowable stress must
     be positive.")
    # N/mm^2 equals MPa; force is signed, criterion uses
     magnitude.
    stress_mpa = force_n / area_mm2
    if not math.isfinite(stress_mpa):
        raise ValueError("Calculated stress is not finite.
     ")
    if abs(stress_mpa) <= allowable_mpa:
        status = "PASS stated rule"
    else:
        status = "FAIL stated rule"
    return stress_mpa, status
```

Complete source

Earlier lines check types and finiteness. This excerpt checks ranges, calculates stress, and selects a status.

Limited claim

PASS means this supplied stress criterion is met, not that a real component is proven safe.

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Integrated test matrix: success and rejection

Input change	Expected result	Reason
Baseline: 10,000 N / 80 mm^2	125 MPa; PASS	known answer and equality
Force = 10,008 N	125.1 MPa; FAIL	just over the criterion
Force = 0 N	0 MPa; PASS	zero-load boundary
Area = 0 or -1 mm^2	`ValueError`	invalid geometry
NaN / infinity / Boolean	documented rejection	invalid numeric-data type/value

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

Lecture 21 exit ticket

1. For a nonpositive area, why raise an error instead of returning zero?
2. When does the optional else of try/except run?
3. Which is appropriate for user input: assert or explicit validation? Why?
4. Write one known-answer check and one expected-error check.

Individual - 3 min

Wang, Ch. 8, Sections 8.10–8.14; Guttag, Ch. 7.

NAZARBAYEV UNIVERSITY

Optional extension: finally is for cleanup

```
# Optional control-flow extension: always
    finish this check's cleanup.
text = "bad"
try:
    value = float(text)
except ValueError:
    print("Cannot convert")
else:
    print("Converted", value)
finally:
    print("Check finished")
```

Optional extension: Python Tutorial, Section 8.7.

Normal completion or exception

Within ordinary Python control flow, finally executes when leaving the try statement.

Do not confuse it with else

else is for a successful try path. finally is not evidence that a result was obtained.

NAZARBAYEV UNIVERSITY

Optional: absolute and relative tolerance

$$|a - b| \leq \max(\varepsilon_{\text{rel}} \max(|a|, |b|), \varepsilon_{\text{abs}}) .$$

```
# Optional: combine a relative tolerance with an
    absolute floor.
import math
actual = 1000000.001
expected = 1000000.0
print(math.isclose(actual, expected,
                   rel_tol=1e-8, abs_tol=1e-9))
print(math.isclose(1e-12, 0.0,
                   rel_tol=1e-8, abs_tol=1e-9))
```

Scale matters

Relative tolerance scales with magnitude. An absolute tolerance provides a meaningful floor near zero.

Choose explicitly

Tolerance values here are illustrative, not universal defaults.

Optional extension: Python math reference, isclose.

NAZARBAYEV UNIVERSITY

Optional: inspect state with a debugger

Pause and inspect

- Set a breakpoint before the suspect calculation.
- Inspect parameters and local variables.
- Step into a function to inspect its body.

Compare with the prediction

- Step over a call to inspect its result.
- Stop at the first unexpected state.
- Rerun the saved failing test after repair.

Tool-independent principle

A debugger is another way to observe the same state that a trace table or labeled printout reveals.

Optional teaching extension to the trace-based debugging workflow in the manuscript.

NAZARBAYEV UNIVERSITY

Quick reference: design tests, then diagnose failures

Need	Use	Ask
Expected behavior	Specification and independent example	What should happen?
Input selection	Typical, boundary, invalid cases	Which behavior classes are missing?
Code paths	Black-box plus glass-box tests	Which branches or exits remain untested?
Fault localization	Traceback, values, types, units	Where is the first wrong state?
Repair evidence	Rerun and retain regression cases	Did another behavior change?

NAZARBAYEV UNIVERSITY

Quick reference: exceptions and assertions

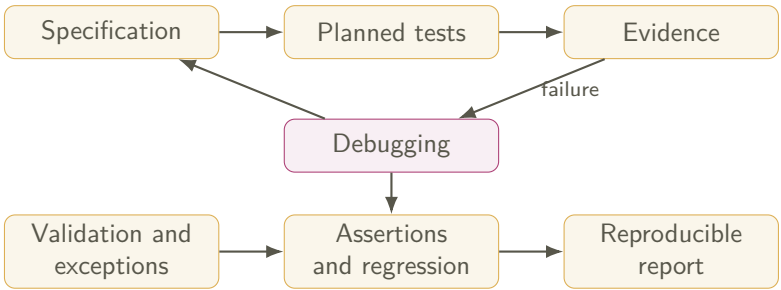
Construct	Role
`raise`	Reject a value or report a failure explicitly
`try / except`	Protect an operation and handle a matching failure
`as error`	Access the exception inside its handler
`else`	Follow up after successful completion of try
`assert`	Check a development or test expectation
`finally`	Optional cleanup on leaving the try statement

Keep failure visible

Validate input explicitly. Do not replace a failed calculation with plausible-looking data.

Wang, Sections 8.10–8.12; Python Tutorial 8.3 and Language Reference 7.3.

Week 7 concept map



NAZARBAYEV UNIVERSITY

Scope, assessment, and the next curriculum unit

Week 7 develops

- Classes of test and testing approaches.
- Debugging steps and error messages.
- Exceptions and assertions.
- Documented engineering evidence.

Later topics

- Classes, attributes, and methods.
- Inheritance and encapsulation.
- Formal algorithmic efficiency.

Assessment authority

Follow the separate Moodle assessment announcements. These instructional examples do not create new assignment requirements or change examination coverage.

ENG 101 Course Specification Report, Sessions 7–10.

NAZARBAYEV UNIVERSITY

Source basis and suggested reading

Course and supplied teaching material

[C] Nazarbayev University, *ENG 101 Course Specification Report*, Fall 2026, Session 7 (CLOs 1, 3, 5).
[W] Y. Wang, *Programming for Engineers with Python*, supplied working draft (file v20260815), Ch. 8, especially Sections 8.2–8.14 and 8.18.
[G] J. V. Guttag, *Introduction to Computation and Programming Using Python*, 2nd ed., Ch. 6–7.

Adaptation note

Examples and diagrams are teaching adaptations or newly constructed exercises within the course scope. Optional additions are labeled. Numerical limits are illustrative unless stated otherwise.

NAZARBAYEV UNIVERSITY

Language reference and preparation

Primary Python references

[P1] Python Tutorial, "Errors and Exceptions," Sections 8.1–8.4 and 8.7.
[P2] Python Language Reference, "The assert statement," Section 7.3.
[P3] Python Standard Library, `math.isfinite` and `math.isclose`.
Official documentation consulted 13 September 2026; links are provided in the README.

Before the next topic

- Run the working examples from a clean session.
- Attempt the debugging starter before opening its solution.
- Explain the test expectation and every caught exception.
- Retain the inputs that exposed a defect.

NAZARBAYEV UNIVERSITY

A trustworthy result comes with evidence

State the contract. Challenge the boundaries. Explain the failure. Repair one cause. Keep the test.

ENG 101 - Programming for Engineers